

Introduction to Programming

COMP101

2025-26

Lab-03

Supervised

Objectives:

Exercise-01	Debugging:	Practical
Exercise-02	BODMAS;	Practical
Exercise-03	Debugging and Test Table:	Reading only
Exercise-04	Walkthrough:	Reading only
Exercise-05:	Test table examples:	Reading only

Practical exercises - we encourage you to attempt these in lab

Reading exercises – not need to be done in lab but may be useful reference points for hints and tips if you start to get unexpected results in your programming.

Exercise-01: Debugging Practical

i) Make a copy of 'COMP101_Lab-03_bugged.py' which is on Canvas and load your copy into IDLE

ii) Debug it to produce an executable program and study the error messages – some explain what has happened, others give a more abstract explanation. It is useful to be able to identify what they really mean.

#COMP101_Lab-03_bugged.py:

i/p two integers and perform subtraction

input #line1

num1 = (input("Enter num1: ")) #line2

num2 = int input("Enter num2 :)) #line3

#calculation #line4

return == (num1+num2) #line5

#output #line6

print (return) #line7

Exercise-01: SOLUTION

Line number:

1. Missing '#' - should be a comment;
Not really a bug: 'input' is okay – but it doesn't do anything
2. Missing 'int' to cast i/p as integer
3. No opening parenthesis after 'int'
5. 'return' is a reserved word;
Can't be used as var name
5. (again) Wrong operator. Should be '=' for assignment
Program uses '==' which is 'equal to'
Wrong operation – should be addition
7. 'return' is a reserved word;
Change it to your var name you used in line5
7. (again) Wrong answer.
It's supposed to be subtraction – see line-5

Code debugged:

#input

```
num1 = int(input("Enter num1: "))  
num2 = int (input("Enter num2: “ ”))
```

#calculation

```
result = (num1-num2)
```

#output

```
print (result)
```

Exercise-02: BODMAS Practical

Sometimes errors in program output are not necessarily due to the coding, but on the use of basic mathematical rules.

Programming calculations in any language still requires sticking to the rules.

Doing calculations requires us to understand the order of precedence between maths operators in the order below: Many web sites give straightforward multiple examples and support if you get stuck. Don't overdo it – sensible use of brackets in calculations often makes things simpler.

Step-1: Check **B**rackets

Step-2: Check **O**perations

Step-3: Check **D**ivision

Step-4: Check **M**ultiplication

Step-5: Check **A**ddition

Step-6: Check **S**ubtraction

(i) Employing BODMAS, what is the answer to:

$$3 + 2 + 1 - 5 + 4\%2 - 1/4 + 6$$

Try it on paper first and see if you get the same answer when you execute it in Python (you could use the shell to test this?)

EXERCISE-02: SOLUTION

Wherever we get several operators at the same level of precedence, the rule is to work left to right:

Green is addition; blue is subtraction; red is division

$$3 + 2 + 1 - 5 + 4\%2 - 1/4 + 6$$

e.g. $3 + 2 + 1$ is done as $3+2 = 5$, then $5+1 = 6$ etc

Step-1: No Brackets involved – move to step-2

Step-2: No operations involved – move to step-3

Step-3: Division – there are two of these

$$3 + 2 + 1 - 5 + 4\%2 - 1/4 + 6$$

Work out: $4\%2$ ($=0$) and $1/4$ ($=0.25$) to give

$$3 + 2 + 1 - 5 + 0 - 0.25 + 6$$

Step-4: No multiplication – move to step-5

Step-5: Addition – there are four of these

Work out: $3 + 2 + 1$ ($=6$), then $(+ 0)$, then $(+6)$ to give

$$12 - 5 + 0 - 0.25$$

Step-6: Subtraction – there are two of these

Work out $12 - 5 - 0.25$ to give:

6.75

$$\text{Solution: } 3 + 2 + 1 - 5 + 4\%2 - 1/4 + 6 = 6.75$$

Exercise-03: Debugging and Test table: Reading only (with some problem solving in it)

(i) Given: The perimeter of a rectangle can be found by summing the length of each of the sides:

The problem has alternative algorithms to solve it:

$$\text{perimeter} = \text{width} + \text{height} + \text{width} + \text{height}$$

OR

$$\text{perimeter} = (2 * \text{height}) + (2 * \text{width})$$

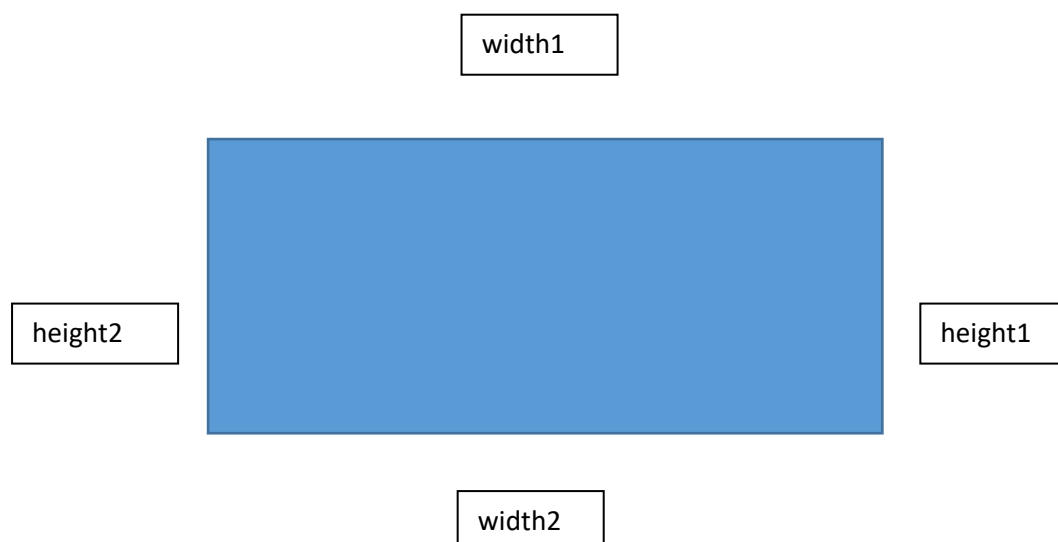
Both of these algorithms are fairly trivial and neither has an advantage over the other, so either one is okay. With deeper problems, we would consider the ‘best one’.

We’ll calculate using:

$$\text{perimeter} = \text{width} + \text{height} + \text{width} + \text{height}$$

Sometimes a ‘picture speaks a thousand words’ – understand what the problem is before coding it.

Do you know what the properties of this rectangle should be?



(ii) Coded solution

#calc perimeter of rectangle from four inputs (no validation)

#rectangle properties - heights must be equal, widths must be equal

#input

```
ht_1 = int(input("Enter height-1: "))
```

```
ht_2 = int(input("Enter height-2: "))
```

```
wd_1 = int(input("Enter width-1: "))
```

```
wd_2 = int(input("Enter width-2: "))
```

#calculate

```
perimeter = (wd_1+ht_1+wd_2+wd_2)
```

#output

```
print(perimeter)
```

(iii) It executes okay: User can input, program can calculate, program can output an answer.

iv) Study the test table.

Test #	height1	height2	width1	width2	Expected	Actual	Comments
1	3	3	4	4	14	15	err
2	2	2	3	3	10	11	err
3	4	4	4	4	16	16	ok
4	2	2	2	2	8	8	ok
5	5	5	1	1	12	8	err

(v) Observations

- We need to debug the program – we have discrepancies between expected and actual. Some inputs work, some don't. Looks like it is okay when heights = widths, but not otherwise.
- All input is valid - we are not considering validation yet: For a rectangle, we expect height1 to equal height2 and width1 to equal width2. Hence the properties for a rectangle are being met.
- We appear to have tested the 'base cases':
(height1 = height2) < (width1 = width2)
(height1 = height2) = (width1 = width2)
(height1 = height2) > (width1 = width2)

vi) **The problem is in line?? It needs amending to ??**

Do a line-by-line analysis with reference to the test table.

Main commentary removed for easier reading.

#input	#line1
ht_1 = int(input("Enter height-1: "))	#line2
ht_2 = int(input("Enter height-2: "))	#line3
wd_1 = int(input("Enter width-1: "))	#line4
wd_2 = int(input("Enter width-2: "))	#line5
 #calculation	 #line6
perimeter = (wd_1+ht_1+wd_2+wd_2)	#line7
 #output	 #line8
print (perimeter)	#line9

Analysis

Line: User i/p

line2: ht_1 3

line3: ht_2 3

line4: wd_1 4

line5: wd_2 4



Input: Okay

It's a rectangle, so two heights should be the same value, and two widths should be the same value

line7: perimeter = 4 + 3 + 4 + 4

Calculation: Okay

Sum four values

line-9: print (15) **#but it should be 14**

Output okay

But wrong answer.

If input is okay and but output is wrong, problem must be in the calculation.

There is a more structured and accurate way to do the above. Expand the test table to analyse each individual I/O. This is a 'walkthrough'.

Exercise-04: Walkthrough: Reading only (with some problem solving in it)

(i) A ‘walkthrough’ is where we manually (on paper) execute each line of code by writing in the values input or output by that line of code. Many programmers use a table as the template for a walkthrough.

It is easier to read if we remove all non-executable code (comments etc) and focus only on the ‘active code’.

Line num

```
ht_1= int(input("Enter height-1: "))
ht_2= int(input("Enter height-2: "))
wd_1= int(input("Enter width-1: "))
wd_2= int(input("Enter width-2: "))
perimeter = (wd_1+ht_1+wd_2+wd_2)
print(perimeter)
```

Test for (ht_1 = ht_2) and (wd_1 = wd_2):

Line num	ht_1	ht_2	wd_1	wd_2	perimeter (wd_1+ht_1+wd_2+wd_2)	print	Comments
1	3						
2		3					
3			4				
4				4			
5					4+3+4+4		
6						15	! should = 14

(ii) You can expand the table a bit to track and trace every data item at each line – the table can get a bit crowded in a larger problem. You can initialise variables to 0 until user inputs a value to it or a calculation is performed.

Same as previous table but initialising to 0:
 Test for $ht_1 = ht_2$ and $wd_1 = wd_2$:

Line num	ht_1	ht_2	wd_1	wd_2	perimeter (wd_1+ht_1+wd_2+wd_2)	Print ()	Comments
1	3	0	0	0	0	0	
2	3	3	0	0	0	0	
3	3	3	4	0	0	0	
4	3	3	4	4	0	0	
5	3	3	4	4	4+3+4+4	0	
6	3	3	4	4	4+3+4+4	15	! should = 14

You should be able to see where the error is?

NOTES:

If you were the tutor who has to assess this:

- What do you think of the var names - are they clear and understandable?
- Is the code suitably commented? Is it over-commented?
 - No need to comment lines that are obvious in what they do. Use comments to explain any ‘tricky bits’ or complicated calculations etc. Use commentary to help the next programmer along who has to maintain your code
- Is the code itself clear (possibly aided with commentary) and could be maintained by another programmer?
- What do you think of the prompts?
- Is the answer correct (eventually)?

Exercise-05: Test table examples: Reading only (with some problem solving in it) with solutions

Test tables:

A test table is a ‘clean’ way to provide evidence of testing and indicating problems: The easiest format is:

Test #	Input-1	Input-2	Expected o/p	Actual o/p	Comments

- a) Use as many columns as needed that match the number of inputs.
- b) Use as many rows as needed to show effective testing for the most common types of input that can be expected.
- c) We want to show how a program works for the majority of ‘normal’ input.
It isn’t necessary to show extreme testing, volume testing etc at this stage – more on testing strategies later in the course.
- d) Start first by manually calculating (or use a ‘guestimate’ if the calculation is particularly complex) and fill in the ‘Expected output’ column
- e) It is okay to leave the comment column blank if expected output is the same as actual output.
Better to use the comment column to note any discrepancies.

5.1 Test Addition – fill in the gaps

```

#exercise_add.py
#addition results test
#input
num1 = int(input("Enter num1: "))
num2 = int(input("Enter num2 : "))

#calculate
result = (num1+num2)

#output
print (result)

```

Test #	num1 (int)	num2 (int)	Expected o/p num1+num2	Actual output	Comments Addition
1	15	3			
2	3	15			
3	-15	3			
4	-15	-3			
5	3	15			
6	3	-15			
7	-3	15			
8	-3	-15			

5.1 SOLUTION:

Test#	Expected output	Actual output
1	18	18
2	18	18
3	-12	-12
4	-18	-18
5	18	18
6	-12	-12
7	12	12
8	-18	-18

These numbers are correct and actual output should match the expected output:
Please check your maths if you do not understand these answers

5.2 Test Subtraction – fill in the gaps

Test #	num1 (int)	num2 (int)	Expected o/p num1-num2	Actual o/p	Comments Subtraction
1	15	3			
2	3	15			
3	-15	3			
4	-15	-3			
5	3	15			
6	3	-15			
7	-3	15			
8	-3	-15			

5.2 SOLUTION:

Test#	Expected output	Actual output
1	12	12
2	18	18
3	-18	-18
4	-12	-12
5	-12	-12
6	-18	-18
7	-18	18
8	12	-12

These numbers are correct and actual output should match the expected output:
Please check your maths if you do not understand these answers

5.3 Test Multiplication – fill in the gaps

Test #	num1 (int)	num2 (int)	Expected o/p num1*num2	Actual o/p	Comments Multiplication
1	15	3			
2	3	15			
3	-15	3			
4	-15	-3			
5	3	15			
6	3	-15			
7	-3	15			
8	-3	-15			

5.3 SOLUTION:

Test#	Expected output	Actual output
1	45	45
2	-45	-45
3	-45	-45
4	45	45
5	45	45
6	-45	-45
7	-45	-45
8	45	45

These numbers are correct and actual output should match the expected output:

Please check your maths if you do not understand these answers

5.4 Test Exponentiation – fill in the gaps

Test #	num1 (int)	num2 (int)	Expected o/p $\text{num1}^{**}\text{num2}$	Actual o/p	Comments Exponent
	3	2		9	
	3	-2		0.11111 11111 11111	
	-3	2		9	
	-3	-2		0.11111 11111 11111	
	2	3		8	
	2	-3		0.125	
	-2	3		-8	
	-2	-3		-0.125	

5.4 SOLUTION:

Answers are supplied in the ‘Actual output’ column.

You may wish to see if you can calculate them manually – just to check the computer is correct?

NOTES:

i) Positive exponents

$$2^3 = 2 \times 2 \times 2 = 8$$

$$2^{-3} = 1/2^3 = (1 / 2 \times 2 \times 2) = 1/8 = 0.125$$

$$-2^3 = (-2 \times -2 \times -2) = -8$$

$$-3^{-2} = 1/3^2 = 1 / 3 \times 3 = 1 / 9 = 0.11$$

etc

ii) Negative exponents have the following property:

$$n^{-m} = 1/n^m \quad \text{OR} \quad 1/n^{-m} = n^m$$

Given: -3^{-4}

In the property: $n^{-m} = 1/n^m$

$$n = -3, -m = -4 \text{ (or } 4)$$

Thus: $-3^{-4} = 1/-3^4$

OR

We can rewrite (-3^{-4}) using a positive exponent by taking the reciprocal of the base (-3) and applying the absolute value of the negative exponent (-4) to the denominator of the fraction like this: $1/-3^4$

Now we can simplify the denominator by evaluating -3^4 which is the same as writing:

$$(-3)(-3)(-3)(-3) \text{ giving us } 81.$$

So our fraction in simplest form is $1/81$

One interesting thing to note here is that raising a negative number by an even numbered exponent gives a positive value (as in our case) and raising a negative number by an odd numbered exponent yields a negative result. (Jordan K.)

5.5 Test True division – fill in the gaps

Test #	num1 (int)	num2 (int)	Expected o/p	Actual o/p	Comments Floor division
	10	3		3.3333333333333335	
	10	-3		-3.3333333333333335	‘-’ not part of 18 space width So 19 spaces
	-10	3		-3.3333333333333335	
	-10	-3		3.3333333333333335	
	3	10		0.3	
	3	-10		-0.3	
	-3	10		-0.3	
	-3	-10		0.3	

5.5 SOLUTION:

Answers are supplied in the ‘Actual output’ column.

Optional: You may wish to see if you can calculate them manually – just to check the computer is correct?

5.6 Test Floor division – fill in the gaps

Test #	num1 (int)	num2 (int)	Expected o/p	Actual o/p	Comments Floor division
	10	3		3	
	10	-3		-4	
	-10	3		-4	
	-10	-3		3	
	3	10		0	
	3	-10		-1	
	-3	10		-1	
	-3	-10		0	

5.6 SOLUTION:

Answers are supplied in the ‘Actual output’ column.

Optional: You may wish to see if you can calculate them manually – just to check the computer is correct?

5.7 Test Modulo division – fill in the gaps

Test #	num1 (int)	num2 (int)	Expected o/p	Actual o/p	Comments
	10	3		1	
	10	-3		-2	
	-10	3		2	
	-10	-3		-1	
	3	10		3	
	3	-10		-7	
	-3	10		7	
	-3	-10		-3	

5.7 SOLUTION:

Answers are in the ‘Actual output’ column.

You may be surprised at some of the Actual output if you have tried the exercise with negative numbers

e.g.

$$-27 \% 13 = 12$$

add 13 to -27 = -14

add 13 to -14 = -1

add 13 to -1 = 12 **#13 does not divide into 12 - stop**

e.g. for a negative value -13

27 % -13 we would subtract 13 from 27 etc rather than add it